Package 'Rsimcity'

June 4, 2026

Type Package

Title R6 Suite for Simulations

Version 0.0.6

Author Marcello Gallucci [aut, cre]

Maintainer Marcello Gallucci <mcfanda@gmail.com>

Description Provides generic function to setup and run simulations and experiments.
Beta version.

Imports ggplot2

Suggests R6, later, shiny, testthat (>= 3.0.0)

Config/testthat/edition 3

License GPL

Encoding UTF-8

Config/roxygen2/version 8.0.0

NeedsCompilation no

Contents

`correlated_sample` *Generate a correlated sample*

Description

Generate a data frame containing ‘N‘ observations from a multivariate normal distribution with a specified correlation matrix, means, and standard deviations.

Usage

```
correlated_sample(N, R, mu = 0, sd = 1, seed = NULL)
```

Arguments

`N`	Integer. Number of observations to generate.
`R`	Numeric matrix. A positive definite correlation matrix. The number of rows and columns defines the number of generated variables.
`mu`	Numeric vector. Means of the generated variables. If a single value is supplied, it is recycled across variables. Default is ‘0‘.
`sd`	Numeric vector. Standard deviations of the generated variables. If a single value is supplied, it is recycled across variables. Default is ‘1‘.
`seed`	Optional integer. Random seed used for reproducible sampling. Default is ‘NULL‘.

Details

The function first generates independent standard normal variables and then applies the Cholesky decomposition of ‘R‘ to induce the requested correlation structure. The variables are then rescaled using ‘sd‘ and shifted using ‘mu‘.

‘R‘ must be a square, symmetric, positive definite matrix.

Value

A data frame with ‘N‘ rows and ‘K‘ columns, where ‘K = nrow(R)‘. Variables are named ‘x1‘, ‘x2‘, ..., ‘xK‘.

Examples

```
R <- matrix(
  c(1.0, 0.5,
    0.5, 1.0),
  nrow = 2,
  byrow = TRUE
)

dat <- correlated_sample(N = 100, R = R, mu = c(0, 2), sd = c(1, 3))
head(dat)
```

```
cor(dat)
```

`count_names` *Count elements*

Description

Count the frequency of elements of a vector in another vector

Usage

```
count_names(V, X)
```

Arguments

`V`	Character vector of values to count
`X`	Vector in which to count occurrences of ‘V‘

`Interactive` *Interactive simulation monitor*

Description

Interactive provides a minimal Shiny-based monitor for visualising simulation output as it is produced. It is intended to be driven by a Runner object (see Rsimcity::Runner): the Runner calls $update() after each replication and $update_agg() after each condition completes. The monitor shows a live plot of the current-condition data and — when an aggregate plotting function is supplied — an accumulating aggregated-results plot.

The Interactive class does not run simulations itself; it only receives and displays results produced elsewhere. This separation keeps the monitor lightweight and suitable for embedding in RStudio or browser-based UIs.

Details

Basic workflow (live plot only):

mon <- Interactive$new(plot_fun = my_plot)

mon$set_start_fun(function(obj) runner$experiment_interactive(interactive = obj, Rep = 100))

mon$run(viewer = "pane")

Workflow with an aggregated-results plot:

mon <- Interactive$new(plot_fun = my_plot, agg_plot_fun = my_agg_plot)

When ‘agg_plot_fun‘ is supplied the Shiny app shows two tabs: **"Live"** displays the current-condition raw data updated every replication; **"Aggregated"** displays the output of the runner’s aggregate pipeline, updated once per completed condition and accumulated across conditions.

The Shiny app provides Start, Stop, Continue, and Reset buttons.

Both plotting functions are called with two positional arguments:

plot_fun(data, step) # ‘data‘ = current-condition accumulated output agg_plot_fun(agg_x, step) # ‘agg_x‘ = aggregated output, all conditions so far

where ‘step‘ is the total number of per-replication updates received.

The ‘viewer‘ argument to ‘$run()‘ controls where the app opens: ‘"pane"‘ (RStudio Viewer), ‘"browser"‘ (system browser), ‘"dialog"‘ (RStudio external window), or ‘"none"‘.

Public fields

`plot_fun` Function. Function used to create the live plot. Called as ‘plot_fun(data, step)‘ where ‘data‘ is the current-condition accumulated output.

`agg_plot_fun` Function or ‘NULL‘. Optional function used to plot aggregated results. Called as ‘agg_plot_fun(agg_x, step)‘ after each condition completes. If ‘NULL‘ (the default), no aggregated plot tab is shown.

`start_fun` Function or ‘NULL‘. Function called when the Start button is pressed.

`x` Object. Accumulated simulation output for the current condition.

`agg_x` Object. Accumulated aggregated output across all completed conditions.

`step` Integer. Number of updates received.

`running` Logical. Whether the monitor is currently running.

`paused` Logical. Whether the monitor is paused.

`finished` Logical. Whether the simulation has finished.

`dirty_plot` Logical. Whether the plot needs to be redrawn.

`redraw_pending` Logical. Whether a plot redraw has already been scheduled.

`run_id` Integer. Internal counter used to invalidate old scheduled callbacks.

`info` List. General-purpose list for user data. Also used internally to store the error message (‘info$error‘) when a step throws an unhandled error during ‘$experiment_interactive()‘.

`condition` Named list or ‘NULL‘. The current design condition being processed, set automatically by [Runner]‘$experiment_interactive()‘.

`title` Character. Title shown in the Shiny app.

`plot_refresh_ms` Numeric. Minimum delay, in milliseconds, between plot redraws.

`plot_height` Character. Height of the Shiny plot output.

`max_rows` Integer or ‘NULL‘. Maximum number of rows to retain when ‘x‘ is a data frame.

Methods

Public methods:

- `simcity_interactive$new()`
- `simcity_interactive$set_start_fun()`
- `simcity_interactive$set_agg_plot_fun()`
- `simcity_interactive$update_agg()`

- simcity_interactive$start()
- simcity_interactive$update()
- simcity_interactive$clear_data()
- simcity_interactive$reset()
- simcity_interactive$pause()
- simcity_interactive$resume()
- simcity_interactive$finish()
- simcity_interactive$is_paused()
- simcity_interactive$is_finished()
- simcity_interactive$is_running()
- simcity_interactive$invalidate_data()
- simcity_interactive$set_condition()
- simcity_interactive$invalidate_plot()
- simcity_interactive$schedule_plot_redraw()
- simcity_interactive$depend_data()
- simcity_interactive$depend_plot()
- simcity_interactive$invalidate_agg_plot()
- simcity_interactive$depend_agg_plot()
- simcity_interactive$app()
- simcity_interactive$run()
- simcity_interactive$clone()

`simcity_interactive$new()`**:** Create a new interactive simulation monitor.

Usage:

```
simcity_interactive$new(
  plot_fun,
  agg_plot_fun = NULL,
  title = "Live Shiny plot",
  plot_refresh_ms = 500,
  plot_height = "500px",
  max_rows = NULL
)
```

Arguments:

`plot_fun` Function used to create the live plot. It is called as ‘plot_fun(data, step)‘, where ‘data‘ is the accumulated simulation output for the current condition and ‘step‘ is the current update number.

`agg_plot_fun` Optional function used to plot aggregated results. It is called as ‘agg_plot_fun(agg_x, step)‘ after each condition completes, where ‘agg_x‘ accumulates the aggregated output across all conditions. When non-‘NULL‘, a second "Aggregated" tab is shown in the Shiny app.

`title` Character. Title displayed in the Shiny app.

`plot_refresh_ms` Numeric. Minimum delay, in milliseconds, between plot redraws.

`plot_height` Character. Plot height passed to [shiny::plotOutput()].

`max_rows` Integer or 'NULL'. If 'x' is a data frame, keep only the last 'max_rows' rows. If 'NULL', keep all rows.

Returns: A new 'Interactive' object.

`simcity_interactive$set_start_fun()`: Register the function called when the Start button is pressed.

Usage:

```
simcity_interactive$set_start_fun(fun)
```

Arguments:

`fun` Function. The function should accept the interactive object as its first argument. For example: 'function(obj) runner$experiment_interactive(obj)'.

Returns: Invisibly returns 'self'.

`simcity_interactive$set_agg_plot_fun()`: Register the optional aggregated-results plotting function.

Usage:

```
simcity_interactive$set_agg_plot_fun(fun)
```

Arguments:

`fun` Function. Called as 'fun(agg_x, step)' after each condition completes. When set, an "Aggregated" tab appears in the Shiny app.

Returns: Invisibly returns 'self'.

`simcity_interactive$update_agg()`: Add new aggregated simulation output to the monitor. Called by [Runner]'$experiment_interactive()' after each condition completes. The data are never cleared between conditions; use '$reset()' to clear.

Usage:

```
simcity_interactive$update_agg(x, append = TRUE)
```

Arguments:

`x` Object. Aggregated results for the just-completed condition.

`append` Logical. If 'TRUE', append to the existing aggregated data. If 'FALSE', replace it.

Returns: Invisibly returns 'self'.

`simcity_interactive$start()`: Start the interactive monitor.

Usage:

```
simcity_interactive$start(reset = TRUE)
```

Arguments:

`reset` Logical. If 'TRUE', reset the stored data before starting.

Returns: Invisibly returns 'self'.

`simcity_interactive$update()`: Add new simulation output to the monitor.

Usage:

```
simcity_interactive$update(x, append = TRUE)
```

Arguments:

`x` Object. New simulation output. If both the existing data and the new data are data frames, they are combined with [base::rbind()].

`append` Logical. If 'TRUE', append 'x' to the existing data. If 'FALSE', replace the existing data.

Returns: Invisibly returns 'self'.

`simcity_interactive$clear_data()`: Clear the current-condition live data.

Clears 'x' so the live plot starts fresh for the next condition. Aggregated data ('agg_x') is **not** cleared; use '$reset()' for that. Called automatically by [Runner]'$experiment_interactive()' between conditions.

Usage:

```
simcity_interactive$clear_data(reset_step = TRUE)
```

Arguments:

`reset_step` Logical. If 'TRUE' (default), reset 'step' to zero.

Returns: Invisibly returns 'self'.

`simcity_interactive$reset()`: Reset the monitor.

Usage:

```
simcity_interactive$reset()
```

Returns: Invisibly returns 'self'.

`simcity_interactive$pause()`: Pause the monitor.

Usage:

```
simcity_interactive$pause()
```

Returns: Invisibly returns 'self'.

`simcity_interactive$resume()`: Resume the monitor after it has been paused.

Usage:

```
simcity_interactive$resume()
```

Returns: Invisibly returns 'self'.

`simcity_interactive$finish()`: Mark the monitor as finished.

Usage:

```
simcity_interactive$finish()
```

Returns: Invisibly returns 'self'.

`simcity_interactive$is_paused()`: Check whether the monitor is paused.

Usage:

```
simcity_interactive$is_paused()
```

Returns: Logical.

`simcity_interactive$is_finished()`: Check whether the monitor is finished.

Usage:

```
simcity_interactive$is_finished()
```

Returns: Logical.

`simcity_interactive$is_running()`: Check whether the monitor is running.

Usage:

```
simcity_interactive$is_running()
```

Returns: Logical.

`simcity_interactive$invalidate_data()`: Invalidate Shiny outputs that depend on status/data information.

Usage:

```
simcity_interactive$invalidate_data()
```

Returns: Invisibly returns 'NULL'.

`simcity_interactive$set_condition()`: Set the current design condition shown in the Shiny panel.

Usage:

```
simcity_interactive$set_condition(condition)
```

Arguments:

`condition` Named list or one-row data frame with the current design condition.

Returns: Invisibly returns 'self'.

`simcity_interactive$invalidate_plot()`: Invalidate Shiny outputs that depend on the plot.

Usage:

```
simcity_interactive$invalidate_plot()
```

Returns: Invisibly returns 'NULL'.

`simcity_interactive$schedule_plot_redraw()`: Schedule a plot redraw.

Usage:

```
simcity_interactive$schedule_plot_redraw(immediate = FALSE)
```

Arguments:

`immediate` Logical. If 'TRUE', schedule the redraw immediately. Otherwise, wait 'plot_refresh_ms' milliseconds.

Returns: Invisibly returns 'NULL'.

`simcity_interactive$depend_data()`: Create a dependency on the data/status trigger.

Usage:

```
simcity_interactive$depend_data()
```

Returns: Invisibly returns 'NULL'.

`simcity_interactive$depend_plot()`: Create a dependency on the plot trigger.

Usage:

```
simcity_interactive$depend_plot()
```

Returns: Invisibly returns 'NULL'.

`simcity_interactive$invalidate_agg_plot()`: Invalidate Shiny outputs that depend on the aggregated plot.

Usage:

```
simcity_interactive$invalidate_agg_plot()
```

Returns: Invisibly returns 'NULL'.

`simcity_interactive$depend_agg_plot()`: Create a dependency on the aggregated plot trigger.

Usage:

```
simcity_interactive$depend_agg_plot()
```

Returns: Invisibly returns 'NULL'.

`simcity_interactive$app()`: Build the Shiny application.

Usage:

```
simcity_interactive$app()
```

Returns: A Shiny application object.

`simcity_interactive$run()`: Run the Shiny app.

Usage:

```
simcity_interactive$run(
  port = NULL,
  viewer = c("pane", "browser", "dialog", "none")
)
```

Arguments:

`port` Integer or 'NULL'. Port passed to [shiny::runApp()].

`viewer` Character. Where to display the app:

- **'"pane"'** RStudio Viewer pane (default).
- **'"browser"'** System web browser.
- **'"dialog"'** RStudio external window (floating dialog).
- **'"none"'** Do not open any viewer.

Returns: Runs the Shiny app.

`simcity_interactive$clone()`: The objects of this class are cloneable with this method.

Usage:

```
simcity_interactive$clone(deep = FALSE)
```

Arguments:

`deep` Whether to make a deep clone.

Examples

```
## Not run:
# Minimal example — Runner drives the simulation and Interactive displays it
runner <- Rsimcity::Runner$new("example")
runner$params <- list(N = 10)
runner$design <- list(mu = c(0, 1))
runner$step <- function(N, mu) data.frame(x = stats::rnorm(N, mean = mu))

mon <- Rsimcity::Interactive$new(plot_fun = function(data, step) NULL)
runner$experiment_interactive(interactive = mon, Rep = 5, delay = 0)

## End(Not run)
```

`plot_by_columns` *Plot columns matching a name pattern against an x variable*

Description

Plots several numeric columns whose names match a given pattern, such as ‘".SE"‘ or ‘".p"‘, against a selected x variable. Values are aggregated by the x variable before plotting.

Usage

```
plot_by_columns(
  data,
  xvar,
  pattern = NULL,
  varsname = NULL,
  fun = mean,
  na.rm = TRUE,
  fixed = FALSE,
  xlabel = NULL,
  ylabel = NULL,
  legeng = TRUE,
  legend_labels = NULL,
  title = NULL,
  line = TRUE,
  points = TRUE,
  colors = NULL,
  linetypes = NULL
)
```

Arguments

`data`	A ‘data.frame‘.
`xvar`	Character string. Name of the variable to use on the x-axis.

pattern	Character string. Pattern used to select columns. Passed to ‘grep()‘. For example, ‘".SE"‘ selects columns such as ‘"po.SE"‘, ‘"ca.SE"‘, and ‘"fu.SE"‘.
varsname	Optional character vector of column names to plot (used if ‘pattern‘ is ‘NULL‘).
fun	Function used to aggregate values over rows with the same value of ‘xvar‘. Default is ‘mean‘.
na.rm	Logical. Should missing values be removed before aggregation? Default is ‘TRUE‘.
fixed	Logical. Passed to ‘grep()‘. If ‘TRUE‘, ‘pattern‘ is matched as plain text rather than as a regular expression. Default is ‘TRUE‘.
xlabel	Optional character string. Label for the x-axis. If ‘NULL‘, ‘xvar‘ is used.
ylabel	Optional character string. Label for the y-axis. If ‘NULL‘, ‘pattern‘ is used.
legeng	Logical. Whether to show a legend (default ‘TRUE‘).
legend_labels	Optional character vector of labels for the legend, same length as matched columns.
title	Optional character string. Plot title.
line	Logical. If ‘TRUE‘, lines are drawn. Default is ‘TRUE‘.
points	Logical. If ‘TRUE‘, points are drawn. Default is ‘TRUE‘.
colors	Optional character vector of colors for the plotted lines/points.
linetypes	Optional character vector of linetypes for the matched columns.

Details

The function identifies all columns whose names match ‘pattern‘, reshapes them internally to long format, aggregates the values by ‘xvar‘ and column name, and then plots one line per matched column.

This is useful for comparing quantities such as different standard errors, p-values, or test statistics stored in separate columns.

Value

A ‘ggplot‘ object.

Examples

```
## Not run:
plot_by_columns(results_ubc, xvar = "N", pattern = ".SE")

plot_by_columns(results_ubc, xvar = "rho", pattern = ".p")

plot_by_columns(
  data = results_ubc,
  xvar = "N",
  pattern = ".SE",
  ylabel = "Standard error",
  title = "Estimated SE by sample size"
)

## End(Not run)
```

plot_by_method *Plot aggregated simulation results by method*

Description

Produces a ‘ggplot2‘ line/point plot of one outcome variable against one predictor variable, separately for each method. The outcome is aggregated across all other variables in the data.

Usage

```
plot_by_method(
  data,
  xvar,
  yvar,
  method_var = "method",
  fun = mean,
  na.rm = TRUE,
  xlabel = NULL,
  ylabel = NULL,
  title = NULL,
  line = TRUE,
  points = TRUE
)
```

Arguments

data	A ‘data.frame‘ containing the simulation results.
xvar	Character string. Name of the variable to use on the x-axis.
yvar	Character string. Name of the numeric variable to plot on the y-axis.
method_var	Character string. Name of the variable identifying the method. Default is ‘"method"‘.
fun	Function used to aggregate ‘yvar‘ over the remaining variables. Default is ‘mean‘.
na.rm	Logical. Should missing values be removed before aggregation? Default is ‘TRUE‘.
xlabel	Optional character string. Label for the x-axis. If ‘NULL‘, ‘xvar‘ is used.
ylabel	Optional character string. Label for the y-axis. If ‘NULL‘, ‘yvar‘ is used.
title	Optional character string. Plot title.
line	Logical. If ‘TRUE‘, lines are drawn. Default is ‘TRUE‘.
points	Logical. If ‘TRUE‘, points are drawn. Default is ‘TRUE‘.

Details

The function first keeps only ‘xvar‘, ‘yvar‘, and ‘method_var‘. It then aggregates ‘yvar‘ by each combination of ‘xvar‘ and ‘method_var‘. Therefore, all other columns in ‘data‘ are averaged over implicitly.

This is useful for simulation summaries where results are computed over several design factors, but the plot should show the average behaviour of each method as a function of a single variable.

Value

A ‘ggplot‘ object.

Examples

```
## Not run:
plot_by_method(simdata, xvar = "N", yvar = "rmse")

plot_by_method(
  data = simdata,
  xvar = "rho",
  yvar = "PPV",
  ylabel = "Positive predictive value"
)

plot_by_method(
  data = simdata,
  xvar = "N",
  yvar = "bias",
  title = "Bias by sample size"
)

## End(Not run)
```

`Runner` *Simulation runner*

Description

‘Runner‘ is an R6 class used to define and run simulation experiments.

A runner stores fixed parameters, experimental design conditions, simulation steps, and optional aggregate functions. The main entry points are:

* ‘$experiment()‘ — runs all design conditions for a given number of replications and returns a data frame of aggregated results. * ‘$experiment_interactive()‘ — same pipeline, but feeds results into an [Interactive] monitor as they are produced, enabling live Shiny plots.

Details

Simulation steps are functions whose formal arguments are matched by name against the parameters stored in ‘params‘, the current design condition, and the output of the previous step, passed as ‘data‘.

If a step function has an argument named ‘simcity‘, the runner object itself is passed to that argument.

Aggregate functions are applied after the replications for one condition have been completed. Their formal arguments are matched in the same way. The last aggregate function must return a [data.frame]. The default aggregate is ‘as.data.frame(do.call(rbind, data))‘. Use ‘obj$aggregate <- FALSE‘ to disable aggregation entirely.

In interactive mode, the aggregate pipeline is also run at the end of each condition and the result is pushed to [Interactive]‘$update_agg()‘, so an optional second plot function can visualise per-condition summaries as they accumulate.

Experiments can be run in parallel using the [future] package. ‘Runner‘ prepares the parallel sessions and collects the results. Functions defined in ‘.GlobalEnv‘ are exported to the parallel sessions, so user-defined functions can be used by the experiment code.

Public fields

`name` Character. Name of the simulation runner.

`debug` Integer. Debugging level. Higher values print more information.

`info` List. Optional list for storing user-defined information.

`parallel` Logical. If ‘TRUE‘, conditions are evaluated in parallel using ‘future::multisession‘ (or ‘future::multicore‘ on Linux); otherwise they are evaluated sequentially. One process is spawned per design cell, so a single-cell design is not parallelised. To parallelise it, replicate the cell, e.g. ‘obj$design <- list(N = c(100, 100, 100))‘ with ‘obj$experiment(Rep = 400)‘.

`par_method` Character. Parallel backend when ‘parallel = TRUE‘. ‘"multisession"‘ (default) works on Windows and Linux; ‘"multicore"‘ works on Linux only.

`collect` List. Names of objects in ‘.GlobalEnv‘ to export to parallel workers. By default all functions in ‘.GlobalEnv‘ are exported; non-function objects (data frames, etc.) must be listed here explicitly. Ignored when ‘parallel = FALSE‘.

Active bindings

`params` Named list. Fixed simulation parameters passed to simulation steps.

`design` Named list. Experimental design parameters. All combinations are expanded with [base::expand.grid()]. In case some parameter is already set in ‘obj$params‘, the ‘obj$design‘ is used and the ‘obj$params‘ parameter is ignored.

`step` Function or list of functions. Adds a simulation step, or returns the currently defined steps when accessed without assignment. obj$step can be called repeatedly to add a sequence of functions to be run. Each function receives the argument ‘data‘ from the previous step If obj$step<-list(a=fun,b=fun) is a list of functions, the functions are run in parallel, each with the previous step ‘data‘ as an argument. If the simcity Runner object is needed by the function, add ‘simcity=‘ argument to the function and then it can be accessed as ‘simcity$foo‘, where ‘foo‘ is any method or variable exposed by the Runner.

`aggregate` Function(s). Adds an aggregate function, or returns the currently defined aggregate functions when accessed without assignment. Can be called repeatedly to chain multiple functions. Each function should accept a 'data' argument; the last one must return a [data.frame]. Aggregating functions are applied sequentially to the data collected by '$step' functions. The default is 'as.data.frame(do.call(rbind, data))'. To disable aggregation, set 'obj$aggregate <- FALSE'. The same pipeline is also run at the end of each condition during '$experiment_interactive()' to feed the aggregated-results plot in [Interactive].

Methods

Public methods:

- `simcity_runner$new()`
- `simcity_runner$methods()`
- `simcity_runner$print()`
- `simcity_runner$experiment()`
- `simcity_runner$experiment_interactive()`
- `simcity_runner$one_cycle()`
- `simcity_runner$test_cycle()`
- `simcity_runner$test_aggregates()`
- `simcity_runner$test_one()`
- `simcity_runner$reset_steps()`
- `simcity_runner$clone()`

`simcity_runner$new()`: Create a new simulation runner.

Usage:

```
simcity_runner$new(name = "SimCityRunner")
```

Arguments:

`name` A label for this object

Returns: A new 'Runner' object.

`simcity_runner$methods()`: Print registered private methods.

Usage:

```
simcity_runner$methods()
```

Returns: Invisibly returns 'NULL'.

`simcity_runner$print()`: Print a short summary of the simulation runner.

Usage:

```
simcity_runner$print(...)
```

Arguments:

`...` Additional arguments, currently ignored.

Returns: Invisibly returns 'self'.

`simcity_runner$experiment()`: Run the full simulation experiment.

Usage:

```
simcity_runner$experiment(Rep = 1, label = NULL, progress = TRUE)
```

Arguments:

`Rep` Integer. Number of replications for each design condition.

`label` Optional character label. Currently unused.

`progress` Logical. If 'TRUE', show a progress bar.

Returns: A data frame with the simulation results aggregated by design condition. The returned object has attributes '"label"' and '"time"'.

`simcity_runner$experiment_interactive()`**:** Run the simulation experiment with live Shiny updates.

Executes the same step and aggregate pipeline as '$experiment()', but feeds results into an [Interactive] monitor as each replication completes rather than collecting everything at the end. The event loop is driven by [later::later()], so the Shiny app remains responsive while the simulation runs.

After all replications for a condition are done, the aggregate pipeline ('$aggregate') is applied to the accumulated data and the result is pushed to 'interactive$update_agg()'. If the [Interactive] object has an 'agg_plot_fun' set, this triggers a redraw of the "Aggregated" tab.

Usage:

```
simcity_runner$experiment_interactive(
  interactive,
  Rep = 1,
  delay = 0,
  reset = TRUE
)
```

Arguments:

`interactive` An [Interactive] object. Receives live updates via '$update()' and per-condition aggregated results via '$update_agg()'.

`Rep` Integer. Number of replications per design condition.

`delay` Numeric. Seconds to wait between replications. Increase for smoother animation; use '0' for maximum throughput.

`reset` Logical. If 'TRUE' (default), call 'interactive$reset()' before starting so previous data is cleared.

Returns: Invisibly returns the 'interactive' object.

`simcity_runner$one_cycle()`**:** Run one simulation condition.

Usage:

```
simcity_runner$one_cycle(Rep = 10, design_params = NULL)
```

Arguments:

`Rep` Integer. Number of replications.

`design_params` Optional named list or one-row data frame containing design parameters for this condition.

Returns: A data frame with one row per successful replication. Failed replications are stored in attribute '"fail"'.

`simcity_runner$test_cycle()`: Test a cycle of runs.

Usage:

```
simcity_runner$test_cycle(Rep = 10)
```

Arguments:

`Rep` Integer. Number of replications used for the test the cycle.

Returns: A data frame returned by cycle function.

`simcity_runner$test_aggregates()`: Test aggregate functions.

Usage:

```
simcity_runner$test_aggregates(Rep = 10)
```

Arguments:

`Rep` Integer. Number of replications used for the test run.

Returns: A data frame returned by the aggregate functions.

`simcity_runner$test_one()`: Run one single simulation.

Usage:

```
simcity_runner$test_one()
```

Returns: The result of one full step sequence.

`simcity_runner$reset_steps()`: Reset step functions.

Usage:

```
simcity_runner$reset_steps()
```

Returns: NULL.

`simcity_runner$clone()`: The objects of this class are cloneable with this method.

Usage:

```
simcity_runner$clone(deep = FALSE)
```

Arguments:

`deep` Whether to make a deep clone.

Examples

```
runner <- Runner$new("example")

runner$params <- list(N = 10)
runner$design <- list(mu = c(0, 1))

runner$step <- function(N, mu) {
  data.frame(x = stats::rnorm(N, mean = mu))
}

runner$aggregate <- function(data) {
  data.frame(mean_x = mean(data$x))
}
```

```
runner$test_one()
runner$test_aggregates(Rep = 5)

## Not run:
# Batch experiment
runner$experiment(Rep = 100)

# Live interactive experiment (requires shiny and later)
mon <- Interactive$new(
  plot_fun = function(data, step) {
    ggplot2::ggplot(data, ggplot2::aes(x = x)) +
      ggplot2::geom_histogram(bins = 30) +
      ggplot2::theme_minimal()
  },
  agg_plot_fun = function(data, step) {
    ggplot2::ggplot(data, ggplot2::aes(x = mu, y = mean_x)) +
      ggplot2::geom_col() +
      ggplot2::theme_minimal()
  }
)
mon$set_start_fun(function(obj) {
  runner$experiment_interactive(interactive = obj, Rep = 100)
})
mon$run(viewer = "pane")

## End(Not run)
```

Index